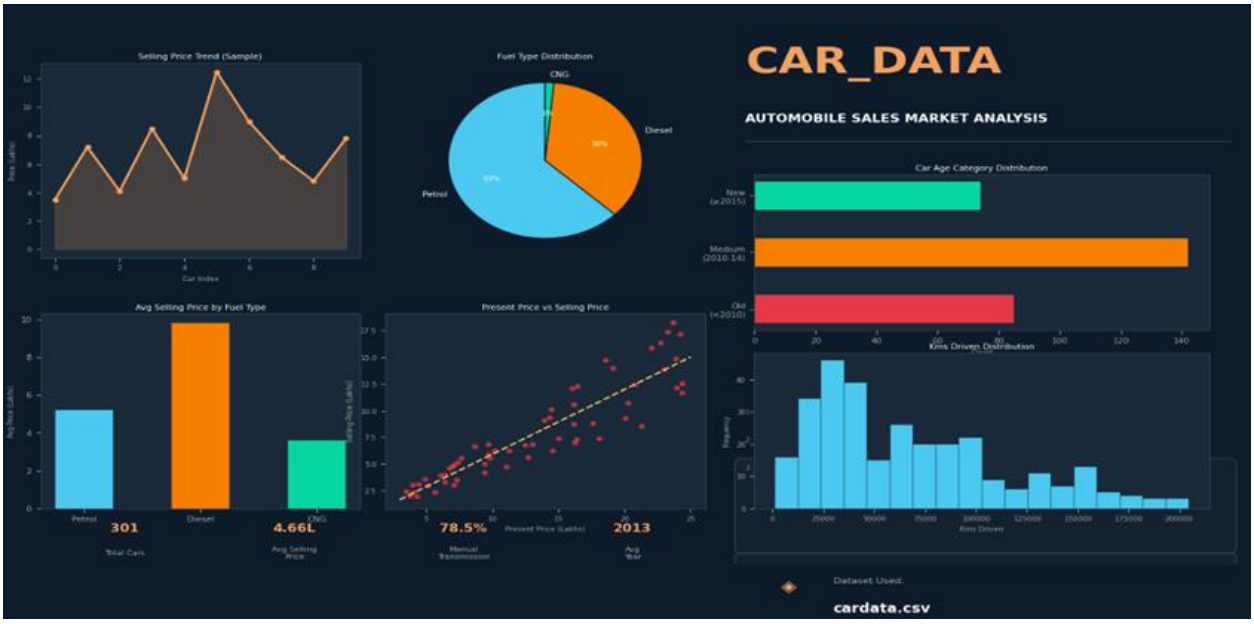


Data Analytics Project: Car Data Analysis

Team members:
1.Harsha Vardhan Gone
2.Namratha Reddy Annem
3.Pradeep Kumar
4.Lavanya Koppula
5.Neeli Ajay Kumar

Dataset Used: cardata.csv

This document outlines a structured, end-to-end data analysis project based on car data. The project is divided into ten scenarios, increasing in difficulty from basic data handling to advanced analysis and visualization, ensuring comprehensive skill coverage.
Data Analytics Project: Car_data_trends.

Dataset: car data dataset (A CSV file)

Project goal: To Analyse car selling prices, fuel type used and kilometres driven pattern across different car categories over time using cardata.csv dataset.

Modules Used:

The **primary libraries/modules** required for this project are:

- **Pandas:** For all data manipulation, filtering, aggregation, and feature engineering tasks.
- **NumPy:** Specifically for numerical calculations like price differences (np.diff).
- **Visualization Library (e.g., Matplotlib):** For generating all required plots (Line Graphs, Bar Charts, Pie Charts, Histograms, Scatter Plots).

🎯 Project Coverage

Category	**Skills Covered**
Data Preprocessing	Data Loading, Missing Value Handling, Data Type Conversion.
Pandas Operations	Filtering, Selection, Aggregation, Feature Engineering
NumPy Calculations	Price difference calculation (np.diff()), Statistical summaries
Visualization	Line Graph, Bar Chart, Pie Chart, Scatter Plot, Histogram

Scenario 1: Data Loading & Basic Cleaning

Description:

In this scenario, we work with a car sales dataset. First, we load the dataset using Pandas. Then, we display the first 5 rows and last 5 rows to understand the data. We check if there are any missing values in important columns such as Selling_Price, Present_Price, Kms_Driven, and Fuel_Type. After that, we fill missing values using mean or mode. Finally, we will convert all numeric columns to proper numeric format.

Task	Description
1.	Load the dataset using Pandas (pd.read_csv()).
2.	Display the first 5 rows (head()) and last 5 rows (tail()), column names (df.columns), and shape of the dataset (df.shape).
3.	Check data types of all columns using df.dtypes
4.	Check for missing values in: Selling_Price, Present_Price, Kms_Driven, Fuel_Type using isnull() function.
5.	Fill missing values: Selling_Price → mean, Present_Price → mean, Kms_Driven → mean, Fuel_Type → mode.
6.	Convert numeric columns to proper numeric types: Selling_Price, Present_Price, Kms_Driven, and Year.(.to_numeric)
7.	Convert Selling_Price and Kms_Driven into NumPy arrays using to_numpy().
8.	Use Numpy to calculate: minimum, maximum, and average selling price.

Output:

```
Displaying First 5 rows:
  Car_Name  Year  Selling_Price  Present_Price  Kms_Driven Fuel_Type Seller_Type Transmission  Owner
0     ritz  2014           3.35           5.59       27000    Petrol      Dealer       Manual      0
1      sx4  2013           4.75           9.54       43000    Diesel      Dealer       Manual      0
2     ciaz  2017           7.25           9.85        6900    Petrol      Dealer       Manual      0
3  wagon r  2011           2.85           4.15        5200    Petrol      Dealer       Manual      0
4    swift  2014           4.60           6.87       42450    Diesel      Dealer       Manual      0
--------------------------------------------------------------------------------
Displaying last 5 rows:
    Car_Name  Year  Selling_Price  Present_Price  Kms_Driven Fuel_Type Seller_Type Transmission  Owner
296     city  2016           9.50           11.6       33988    Diesel      Dealer       Manual      0
297     brio  2015           4.00            5.9       60000    Petrol      Dealer       Manual      0
298     city  2009           3.35           11.0       87934    Petrol      Dealer       Manual      0
299     city  2017          11.50           12.5        9000    Diesel      Dealer       Manual      0
300     brio  2016           5.30            5.9        5464    Petrol      Dealer       Manual      0
--------------------------------------------------------------------------------
Column names:
Index(['Car_Name', 'Year', 'Selling_Price', 'Present_Price', 'Kms_Driven',
       'Fuel_Type', 'Seller_Type', 'Transmission', 'Owner'],
      dtype='object')

Shapes of Dataset:(301, 9)
```

```
--------------------------------------------------------------------------------
Minimun Selling Price: 0.1
Maximum Selling Price: 35.0
Average Selling Price: 4.66
```

Conclusion:

In this scenario, we cleaned the dataset step by step. We handled missing values properly using mean and mode. We also checked and fixed data types and also calculated minimum, maximum and average Selling Price. Now the dataset is clean and ready for analysis.

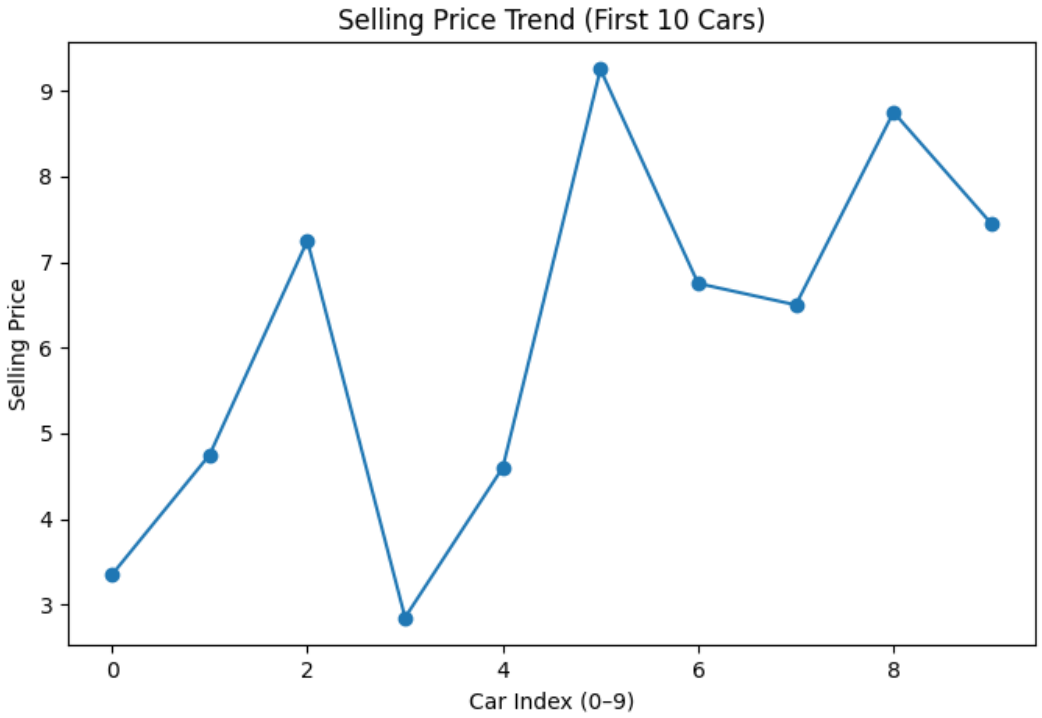
Scenario 2: Selling Price Trend (Line Graph)

Description:

In this scenario, we study how the selling prices of cars change across a small sample. We select only the required columns from the dataset and focus on the first 10 records. The selling prices are converted into a NumPy array for easier handling. A line graph is created using Matplotlib to visualize the trend. This helps us understand how prices vary between different cars.

Task	Description
1.	Select Car_Name and Selling_Price columns.
2.	Take the first 10 rows only using Pandas.
3.	Convert Selling_Price into a NumPy array.
4.	Plot a line graph using Matplotlib: X-axis → row index (0–9), Y-axis → Selling Price, with title, axis labels, and markers.
5.	Save the graph with a suitable filename.

Graph:



The line graph shows the selling price of the first 10 cars. The X-axis represents the car index from 0 to 9, and the Y-axis represents the selling price. Each point on the graph shows the price of a car, and the line connects them to show the trend. We can see that the prices go up and down, which means different cars have different values.

Conclusion:

From this graph, we understand that car selling prices are not constant and vary for each vehicle. Some cars have higher prices, while others are lower. The line graph makes it easy to compare these values visually. This simple analysis helps in identifying price patterns in a small dataset. It is a useful step before doing deeper data analysis.

Scenario 3: Expensive Cars Analysis (Filtering + Bar Chart)

Description:

This scenario is mainly focusing on filtering the dataset to include only cars with a Selling Price which will be greater than 10 to analyze expensive vehicles. The filtered data is grouped by Fuel type for the distribution of different fuel categories. The number of cars in each fuel type is counted and converted into NumPy arrays for better visualization. A bar chart is plotted using Matplotlib with proper labels and title, and the graph is saved for future use.

Task	Description
1.	Filter cars where Selling_Price > 10.
2.	Group the filtered data by Fuel_Type and count number of cars in each fuel type.
3.	Convert fuel type labels and counts into NumPy arrays.
4.	Plot a bar chart using Matplotlib: X-axis → Fuel Type, Y-axis → Count of expensive cars, with title and labels.
5.	Save the graph.

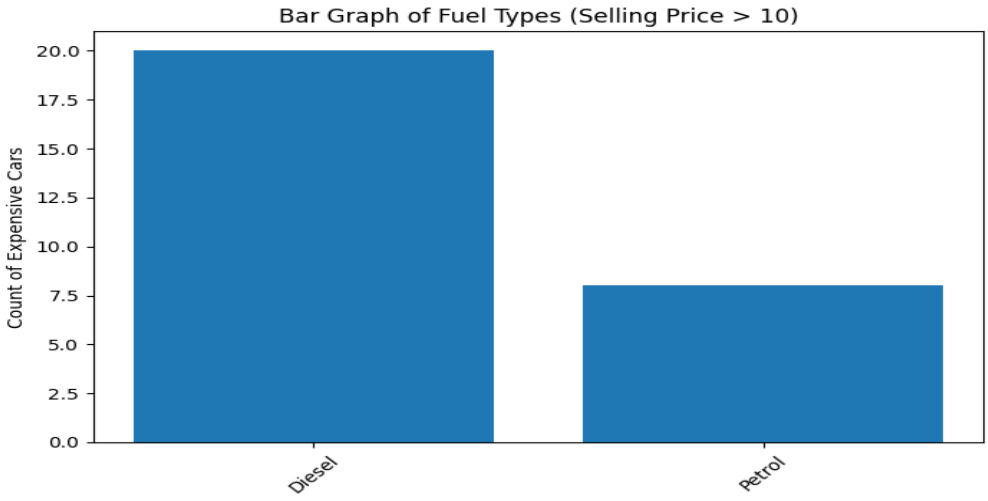
Output: Here, I am filtering cars, Selling_price >10 to focus on expensive vehicles and then grouping the data by Fuel Type to analyze the distribution of fuel categories. Next, I count the number of cars in each fuel type to identify the common ones. Finally, I observe that Diesel cars are more frequent than Petrol among expensive cars.

```
---------- RESTART: C:/pythontasks/Data_Analytics/Project-5/main.py ----------
(S3) Filtering cars with Selling_Price > 10
          Car_Name  Year  Selling_Price  ...  Seller_Type  Transmission Owner
50        fortuner  2012          14.90  ...       Dealer     Automatic     0
51        fortuner  2015          23.00  ...       Dealer     Automatic     0
52          innova  2017          18.00  ...       Dealer     Automatic     0
53        fortuner  2013          16.00  ...   Individual     Automatic     0
59        fortuner  2014          19.99  ...       Dealer     Automatic     0
62        fortuner  2014          18.75  ...       Dealer     Automatic     0
63        fortuner  2015          23.50  ...       Dealer     Automatic     0
64        fortuner  2017          33.00  ...       Dealer     Automatic     0
66          innova  2017          19.75  ...       Dealer     Automatic     0
69   corolla altis  2016          14.25  ...       Dealer        Manual     0
79        fortuner  2012          14.50  ...       Dealer     Automatic     0
80   corolla altis  2016          14.73  ...       Dealer        Manual     0
82          innova  2017          23.00  ...       Dealer     Automatic     0
83          innova  2015          12.50  ...       Dealer        Manual     0
86    land cruiser  2010          35.00  ...       Dealer        Manual     0
91          innova  2014          11.25  ...       Dealer        Manual     0
93        fortuner  2015          23.00  ...       Dealer     Automatic     0
96          innova  2016          20.75  ...       Dealer     Automatic     0
97   corolla altis  2017          17.00  ...       Dealer        Manual     0
211        elantra  2015          11.75  ...       Dealer        Manual     0
212          creta  2016          11.25  ...       Dealer        Manual     0
232        elantra  2015          11.45  ...       Dealer     Automatic     0
237          creta  2015          11.25  ...       Dealer        Manual     0
250          creta  2016          12.90  ...       Dealer        Manual     0
256           city  2016          10.25  ...       Dealer        Manual     0
275           city  2016          10.90  ...       Dealer     Automatic     0
289           city  2016          10.11  ...       Dealer        Manual     0
299           city  2017          11.50  ...       Dealer        Manual     0

[28 rows x 9 columns]
==================================================
(S3) Count of cars per Fuel_Type
 Fuel_Type
Diesel    20
Petrol     8
Name: count, dtype: int64
```

Graph:

Bar chart: It displays data using rectangular bars, where the height or length of each bar represents the value of a category. It is useful for comparing different groups side by side—for example, comparing the number of expensive cars by fuel type (such as diesel vs petrol) based on a specific condition like selling price greater than 10.



Conclusion:

The graph shows that number of expensive cars may varies by fuel type. Diesel cars make up larger portion of the expensive category, while as petrol cars are few in comparison. This indicates that diesel vehicles are most commonly found among higher-priced cars in the dataset. The bar chart makes it easy to compare the counts and understand which fuel type dominates among expensive cars.

Scenario 4: Fuel Type Distribution (Pie Chart)

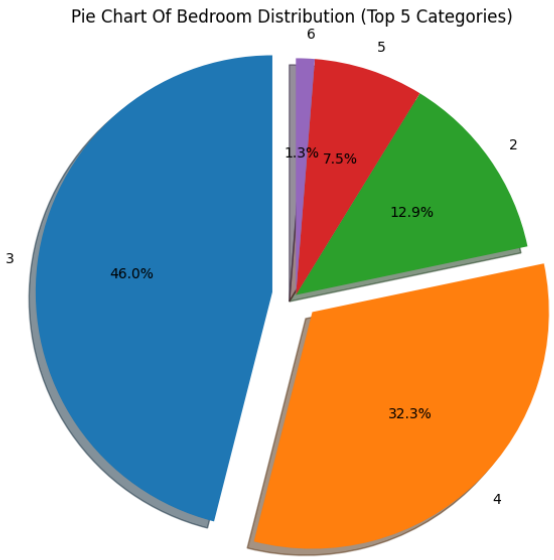
Description:

This analysis examines how cars are distributed based on their fuel type using the dataset. It counts the number of vehicles in each category such as Petrol, Diesel, and CNG. The data is then organized into labels and values for visualization. A pie chart is created to represent the proportion of each fuel type. Percentage labels are added to make the distribution easier to understand. The final graph is saved for reporting and further use.

Task	Description
1.	Count the number of cars in each Fuel_Type category.
2.	Select all categories or top categories if needed.
3.	Prepare labels and values; convert values into a NumPy array.
4.	Plot a pie chart using Matplotlib with percentage labels and title.
5.	Save the graph.

Graph:

This pie chart shows the distribution of houses based on the number of bedrooms. It highlights the top 5 bedroom categories and their percentage share in the dataset. Three-bedroom houses dominate the distribution, followed by four-bedroom homes. Other categories like two, five, and six bedrooms contribute smaller portions.



Conclusion:

The pie chart clearly shows the dominance of certain fuel types in the dataset. It helps identify which fuel type is most commonly used and which ones are less popular. This insight is useful for understanding market trends and consumer preferences in the automobile sector. The visualization makes it easier to compare proportions at a glance. Overall, the analysis provides a simple yet effective summary of fuel usage distribution.

Scenario 5: Present Price vs Selling Price (Scatter Plot)

Description:

In this scenario, we analyze the relationship between the present price and selling price of cars using a scatter plot. First, we select the columns Present_Price and Selling_Price from the dataset. Then, we handle missing values to ensure clean data for analysis. A smaller sample of the data, such as the first 50 rows, is taken for simplicity. Both columns are converted into NumPy arrays for plotting. Finally, a scatter plot is created with proper title and labels to clearly show how present price and selling price are related.

Task	Description
1.	Select Present_Price and Selling_Price columns.
2.	Remove missing values if any.
3.	Take a smaller sample (first 50 or 100 rows) using Pandas.
4.	Convert both columns into NumPy arrays.
5.	Plot a scatter plot using Matplotlib: X-axis → Present_Price, Y-axis → Selling_Price, with title and labels.
6.	Observe whether there is a positive relationship and save the graph.

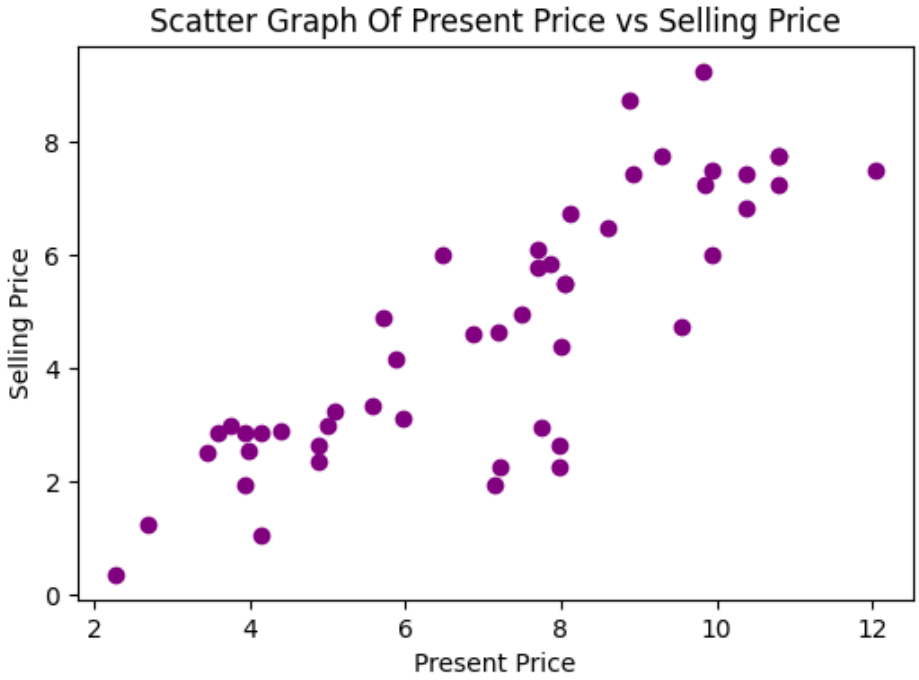
Output Screen: This output displays the Present_Price and Selling_Price columns from the dataset. It creates a new DataFrame named data containing only these two columns for further analysis.

```
(S5)Selecting required coiumns
     Present_Price  Selling_Price
0             5.59           3.35
1             9.54           4.75
2             9.85           7.25
3             4.15           2.85
4             6.87           4.60
..             ...            ...
296          11.60           9.50
297           5.90           4.00
298          11.00           3.35
299          12.50          11.50
300           5.90           5.30

[301 rows x 2 columns]
```

Graph:

- **Scatter plot:** A scatter plot is a graph that uses dots to show the relationship between two numerical variables. Each dot represents one data point, helping us see patterns or relationships between them.



Conclusion:

From this, we can conclude that there is a positive relationship between present price and selling price. Cars with higher present prices are generally sold at higher prices. This shows that expensive cars tend to keep more value even after being used.

Scenario 6: Car Age Category Analysis + Bar Chart

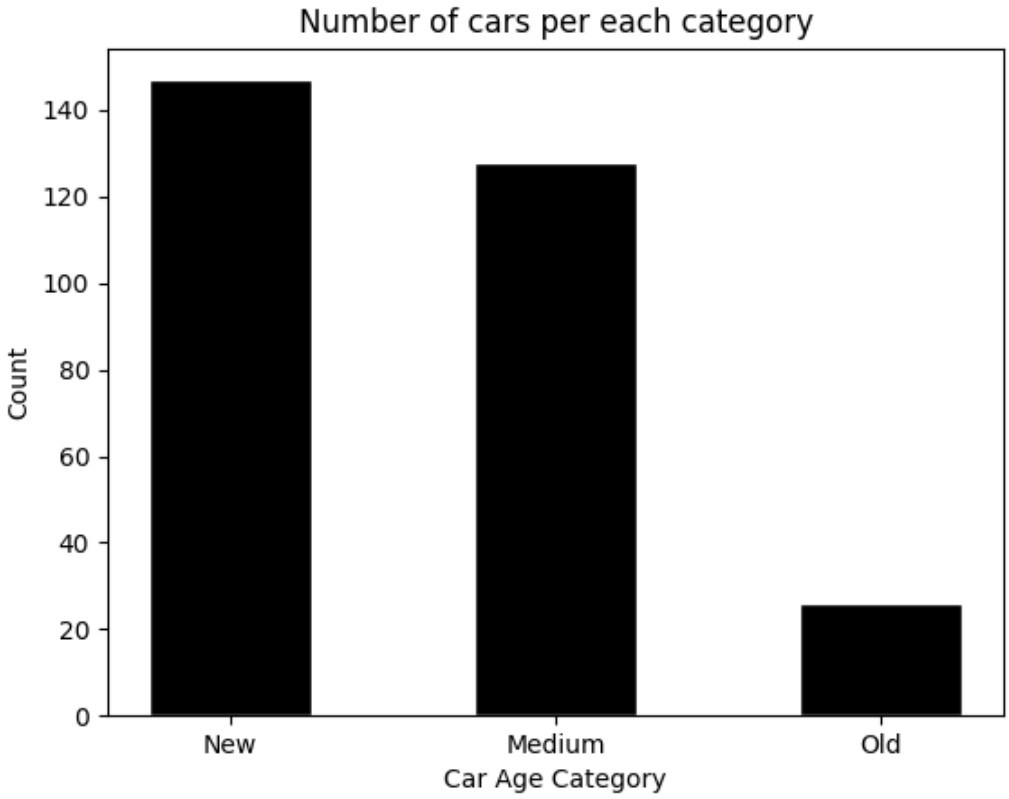
Description:

In this scenario, we create a new feature called Car Age Category using the Year column. Cars are grouped into three categories New, Medium, and Old based on their manufacturing year. This helps us understand how the dataset is distributed across different age groups. We then count the number of cars in each category and visualize the results using a bar chart.

Task	Description
1.	Create a new column: Car Age Category: Year >= 2015 → 'New', 2010 to 2014 → 'Medium', <2010 → 'Old'.
2.	Count number of cars in each Car Age Category.
3.	Convert category names and counts into NumPy arrays.
4.	Plot a bar chart using Matplotlib: • X-axis → Car Age Category, • Y-axis → Count, • Add title and labels.
5.	Save the graph using plt.savefig()

Graph:

The bar chart displays three categories on the X-axis Old, Medium, and New and the count of cars on the Y-axis. The chart shows that the New category (2015 and above) has the highest number of cars, meaning most cars were manufactured after 2015. The Medium category (2010-2014) has a moderate count, while the Old category (before 2010) has the least number of cars. This suggests the dataset is mostly made up of relatively recent cars.



Conclusion:

In this scenario, we successfully created a new feature called Car Age Category using Pandas. We found that most cars in the dataset belong to the New age group, manufactured after 2015. The bar chart clearly shows the distribution across all three categories. This kind of feature engineering helps in better understanding and comparing car data based on age groups, and can be useful for further analysis.

Scenario 7: Kms Driven Distribution (Histogram)

Description:

This scenario mainly focuses on understanding how cars are distributed based on the kilo-meters driven. The Kms Driven column is selected from the dataset and converted it into a NumPy array for analysis part. A histogram is plotted using Matplotlib to visualize the frequency distribution of kilo-meters driven among cars.

Task	Description
1.	Select the Kms_Driven column and convert it into a NumPy array.
2.	Plot a histogram using Matplotlib: X-axis → Kms Driven, Y-axis → Frequency, with a suitable number of bins.
3.	Add title, x-label, and y-label.
4.	Save the graph.
5.	Observe whether most cars have lower or higher mileage.

Output:

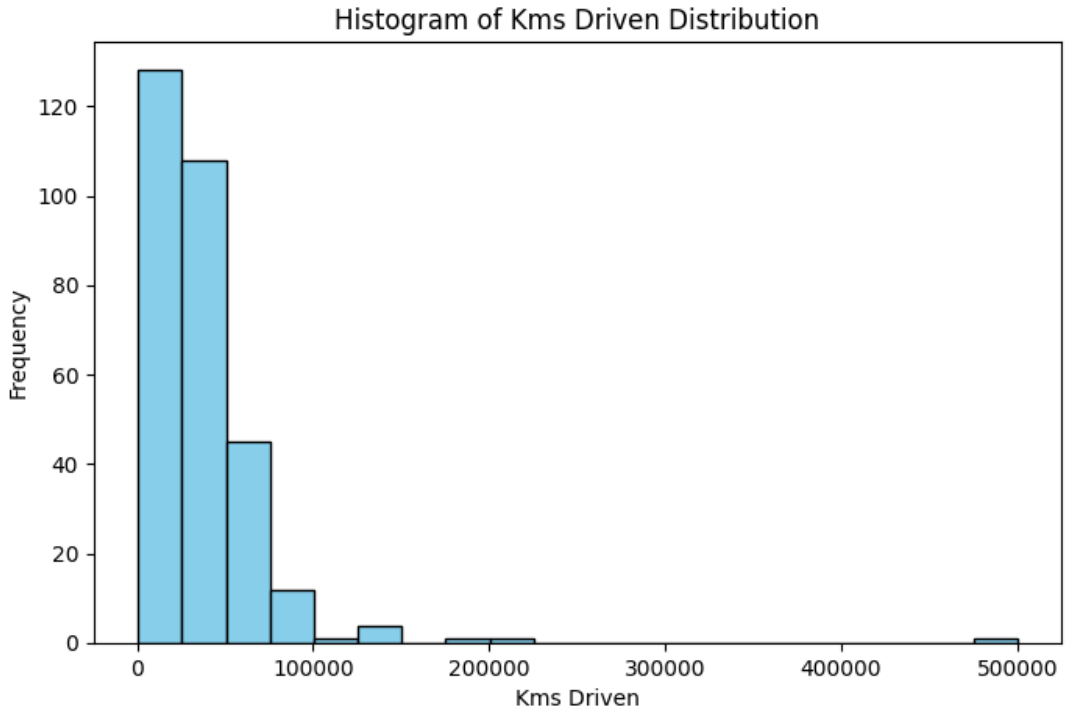
Here, I am selecting the Kms Driven column to analyse how cars are distributed and also converting it into a NumPy array for better visualization. Then, I plot a histogram to observe the frequency distribution of mileage across different ranges. Finally, I observe that most cars fall under lower mileage ranges, indicating that fewer cars have very high kilo-meters driven.

```
============================================================
(S7) Selected Kms_Driven column
 0    27000
1    43000
2     6900
3     5200
4    42450
Name: Kms_Driven, dtype: int64
============================================================
Observation: Most cars tend to have lower mileage if bars are higher on the left side of the histogram.
```

Graph:

Histogram:

It displays the continuous data using bars, where each bar represents a range (bin) of values and its height shows the frequency of data within that range. In this scenario, the histogram helps to identify whether most cars have low or high mileage by observing where the frequency is concentrated or not.



Conclusion:

The histogram shows that how cars are distributed based on kilo-meters driven and observed and also concentrated in lower to mid mileage ranges, while few cars fall into the higher mileage categories. This indicates that a large number of cars in the dataset have been driven less, and small portion have very high usage. The histogram is used to understand the overall spread and frequency of mileage, helping to identify whether low or high driven cars dominate in the dataset.

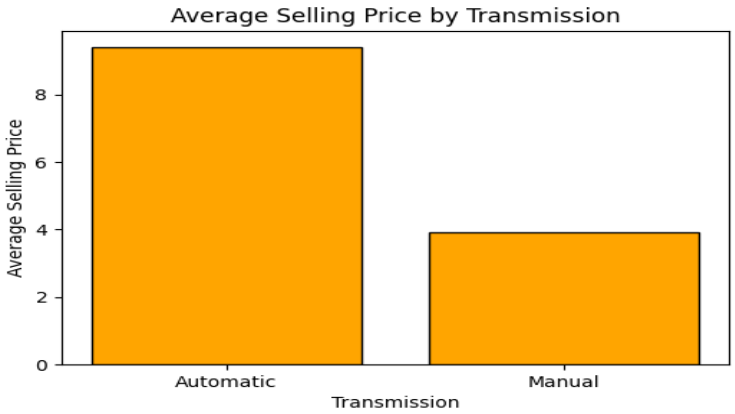
Scenario 8: Transmission-wise Selling Price Comparison

Description:

This scenario analyzes how the selling price of vehicles varies based on their transmission type (e.g., Manual vs Automatic). By grouping the dataset by transmission, we compute the average selling price for each category. This helps identify pricing trends and customer preferences. The results are visualized using a bar chart for easy comparison. Such insights can support pricing strategies and inventory decisions.

Task	Description
1.	Group data by Transmission and calculate average Selling_Price.
2.	Convert transmission labels and average prices into NumPy arrays.
3.	Plot a bar chart using Matplotlib: X-axis → Transmission, Y-axis → Average Selling Price.
4.	Add title and labels.
5.	Save the graph.

Graph:



The bar chart displays transmission types on the X-axis (Manual and Automatic) and their corresponding average selling prices on the Y-axis. Typically, automatic cars tend to have a higher average selling price compared to manual ones, reflecting added convenience and features.

Conclusion:

The analysis shows a clear difference in average selling prices between transmission types. Automatic vehicles generally command higher prices due to their ease of use and growing demand. Manual cars, while more affordable, may appeal to budget-conscious buyers. This comparison helps dealerships and sellers better understand market segmentation. It can also guide customers in making informed purchasing decisions.

Scenario 9: Seller Type Analysis

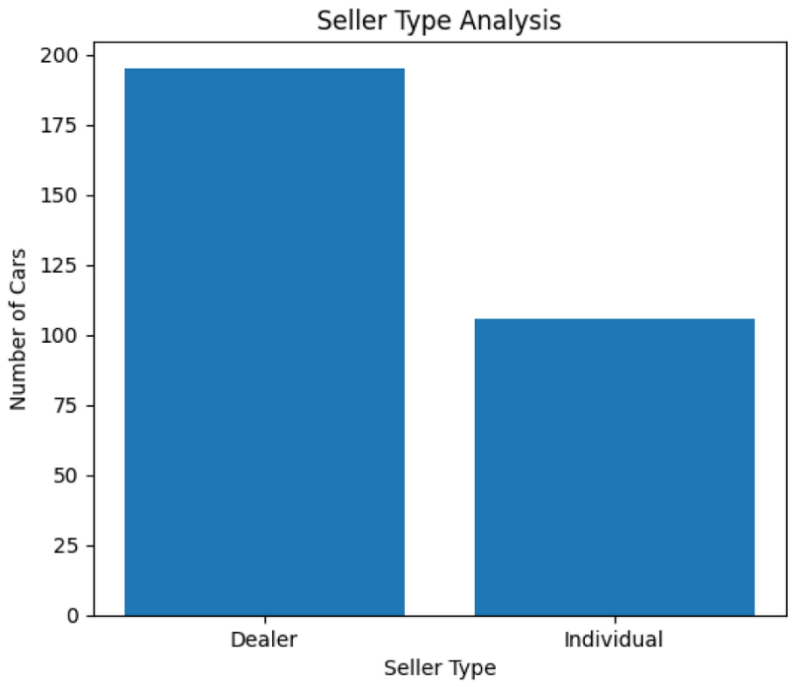
Description:

This analysis compares the number of cars sold by different types of sellers, mainly dealers and individuals. Using Pandas and NumPy, the seller data was counted and organized properly. A bar chart was created with Matplotlib to clearly show the difference between seller types. The graph was also saved so it can be used later in reports or presentations.

Task	Description
1.	Count number of cars by Seller_Type.
2.	Convert results into NumPy arrays.
3.	Plot a bar chart or pie chart using Matplotlib with labels and title.
4.	Save the graph.
5.	Identify which seller type is more common.

Graph:

The bar chart shows the comparison between different seller types based on the number of cars sold. The x-axis represents the seller categories, while the y-axis represents the total number of cars. Each bar indicates how many cars were sold by that seller type. From the graph, it is easy to visually identify which seller category has the highest sales count.



Conclusion:

From the chart, we can easily understand which seller type is more common in the dataset. The seller category with the highest count represents the major source of car sales. This analysis helps in understanding selling patterns in the car market. Visual representation also makes the comparison simple and easy to understand.

Scenario 10: Advanced Analysis + Multiple Graphs

Description:

This analysis performs advanced exploration of car sales data using Pandas, NumPy, and Matplotlib. A new column called Price Difference was created to measure how much value each car lost over time. NumPy functions were used to calculate depreciation statistics and price changes between cars. Different graphs such as line charts, bar charts, and histograms were created to better understand selling price patterns.

Part 1: Feature Creation

Create a new column:

Price Difference

- Price Difference = Present_Price - Selling_Price

This shows how much value the car has depreciated.

Part 2: NumPy Usage

Task	Description
1.	Convert Selling_Price into a NumPy array.
2.	Use NumPy to calculate price changes between consecutive rows using np.diff().
3.	Convert Price Difference column into a NumPy array.
4.	Find: average depreciation, maximum depreciation, minimum depreciation.

Part 3: Visualizations

Line Graph

- Plot Selling_Price trend for all cars.

Bar Chart

- Show average Selling_Price by Fuel_Type.

Histogram

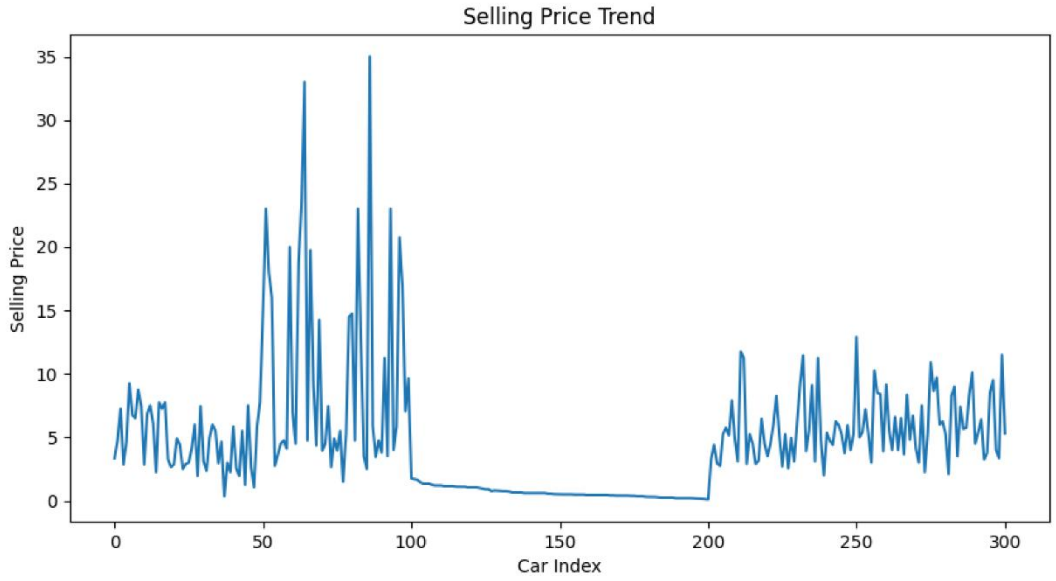
- Plot distribution of Selling_Price.

Part 4: Insights

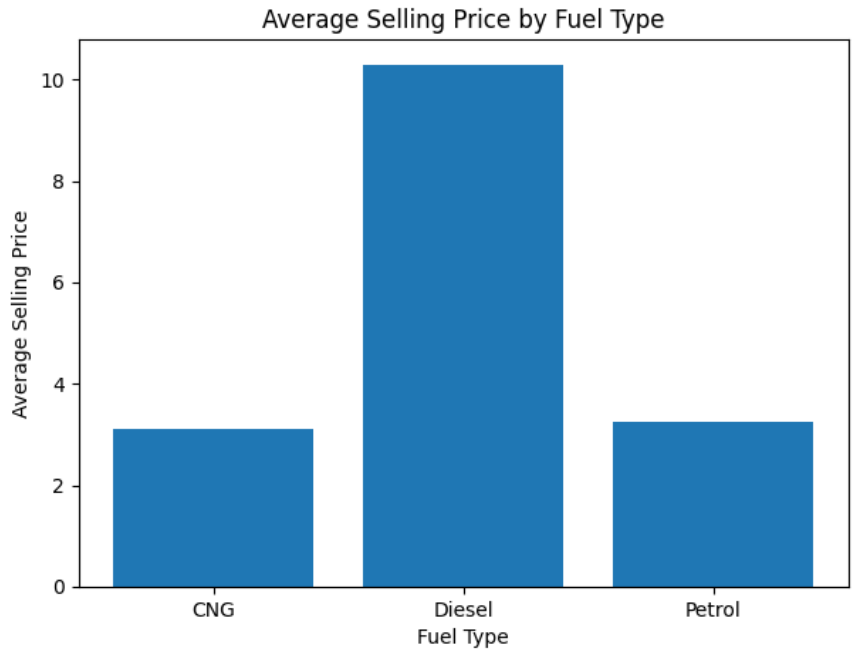
Question	Your Answer
1.	Which fuel type has the highest average selling price?
2.	Which transmission type has higher average selling price?
3.	Are most cars concentrated in lower selling prices or higher selling prices?
4.	Do older cars tend to have lower selling prices?

Graph:

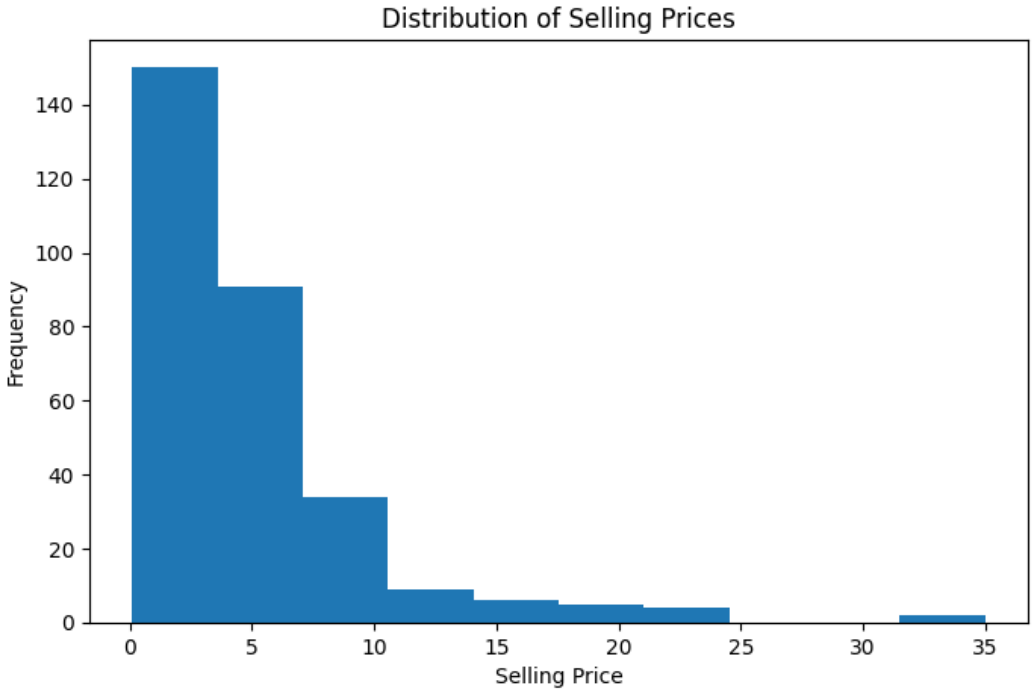
Line graph: The line graph shows the trend of selling prices across all cars in the dataset. The bar chart compares the average selling prices for different fuel types, helping identify which fuel category has higher value. The histogram displays how selling prices are distributed and shows whether most cars fall into lower or higher price ranges. These visualizations make the analysis easier to understand.



Bar graph:



Histogram:



Conclusion:

From the analysis, we can identify the fuel type and transmission type with the highest average selling price. Most cars are generally concentrated in lower selling price ranges based on the histogram distribution. The depreciation analysis also shows how car values decrease over time. In most cases, older cars tend to have lower selling prices compared to newer cars.ded into ten scenarios, increasing in difficulty from basic data handling to advanced analysis and visualization, ensuring comprehensive skill coverage.

Final Conclusion:

This project helped in analyzing car sales data using Pandas, NumPy, and Matplotlib. Different scenarios were used to understand selling prices, fuel types, seller types, transmission types, and depreciation of cars. Various graphs such as bar charts, line graphs, pie charts, scatter plots, and histograms made the data easier to understand visually. The analysis showed important patterns like price distribution, average selling prices, and the effect of car age on selling price. Overall, the project provided a clear understanding of data analysis and visualization techniques in Python.

What you have learned

By doing this project, I learned how to clean, analyze, and visualize real-world data using Python libraries like Pandas, NumPy, and Matplotlib. I understood how to create new features, perform calculations using NumPy arrays, and identify useful insights from datasets. I also learned how different types of graphs help represent data in a simple and meaningful way. This project improved my understanding of data handling, visualization, and basic analytical thinking using Python